

JS index.js X +

```
1 console.log(0.1 * 3 === 0.3);
2 console.log(9007199254740993);
3
4 //Saida: false 9007199254740992
5 //Nunca. O JavaScript não gera erro nem aviso em nenhum dos dois casos.
6 //O resultado fica silenciosamente errado.
```

Console

I/O

AI Agent

159 ms

```
false
9007199254740992
```

main.py

```
1 p = "maçã"
2 print(len(p), len(p.encode()))
3
4 //Saida: 4 6
5 //Nunca. Não há erro nenhum: são duas medidas diferentes, e as duas estão corretas.
```

Console

I/O

AI Agent

180 ms

4 6

main.go

```
1 var b byte = 255
2 b++
3 fmt.Println(b)
4
5 //Saida: main.go:1:1: expected 'package', found 'var'
6 //Nunca. byte é uint8, com valores de 0 a 255. O b++ sai do limite e dá a volta (wraparound) para 0.
7 //O Go não gera erro nem panic nesse caso. O overflow aritmético de inteiros em variáveis é silencioso,
8 //e isso é comportamento definido pela especificação.
9
```

Console

I/O

AI Agent

Exit 1

main.go:1:1: expected 'package', found 'var'

Debug this error

Main.java

```
1 int[] v = new int[3];
2 System.out.println(v[0]);
3 System.out.println(v[3]);
4
5 //Saida: 0
6 //Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 3 out of bounds for length 3
7 //at Main.main(Main.java:5)
8 //(b) Na execução. O compilador não verifica índices de array; ele só confere tipos e sintaxe.
9 //Por isso o código compila normalmente. Durante a execução, a JVM checa os limites a cada acesso.
```

Console

I/O

AI Agent

Exit 1

Main.java:2: error: class, interface, annotation type, enum, record, method or field expected

System.out.println(v[0]);

^

Main.java:3: error: class, interface, annotation type, enum, record, method or field expected

System.out.println(v[3]);

^

2 errors

Debug this error

HelloWorld.rs ×

```
1 let s = String::from("oi");
2 let t = s;
3 println!("{}", s, t);
4
5 //Saida: Nenhuma. O programa não compila.
6 //Na compilação. String guarda dados no heap e não implementa Copy.
7 //Por isso, let t = s; move a posse (ownership) do valor para t, e s deixa de ser válida.
8 //O verificador de empréstimos (borrow checker) proíbe usar s depois do move. Sem essa regra,
9 //haveria dois donos do mesmo dado no heap, com risco de double free.
```

Console

I/O

AI Agent

Exit 1

```
error: expected item, found keyword `let`
--> HelloWorld.rs:1:1
|
1 | let s = String::from("oi");
|   ^^^
|
| `let` cannot be used for global variables
| help: consider using `static` or `const` instead of `let`
|
= note: for a full list of items that can appear in modules, see
<https://doc.rust-lang.org/reference/items.html>
```

error: aborting due to 1 previous error

Debug this error

C Main.c x +

```
1 union { int i; float f; } u;
2 u.f = 1.0f;
3 printf("%d\n", u.i);
4
5 //Saída: (com o código corrigido abaixo): 1065353216
6 //Nunca. Numa union, todos os membros ocupam a mesma região de memória.
7 //Gravar em u.f e ler u.i não converte o valor.
8 //O programa reinterpreta os mesmos 32 bits como inteiro (type punning), sem nenhum erro ou aviso.
```

Console

I/O

AI Agent

Exit 1

Main.c:2:2: error: expected '=', ',', ';', 'asm' or '__attribute__' before
'.' token

```
2 | u.f = 1.0f;
  | ^
```

Main.c:3:8: error: expected declaration specifiers or '...' before string
constant

```
3 | printf("%d\n", u.i);
  |           ~~~~~
```

Main.c:3:16: error: expected declaration specifiers or '...' before 'u'

```
3 | printf("%d\n", u.i);
  |                   ^
```

Debug this error